

DSC630 8.2 Assignment Ayden Trusty

February 15, 2026

1 8.2 Assignment: Time Series Modeling

1.1 DSC630 Ayden Trusty

===== */ Title: Assignment 8.2 Author(s): Abbott, Dean. Applied Predictive Analytics (Textbook) Date: 14 February 2026 Modified By: Ayden Trusty Description: This program is to refresh my understanding/practice of using Python and/or R for time series modeling.

```
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

At this point, I just assume I'm going to be using these packages.

```
[9]: df = pd.read_csv("us_retail_sales.csv")

df.head()
```

```
[9]:  YEAR      JAN      FEB      MAR      APR      MAY      JUN      JUL      AUG  \
0  1992  146925  147223  146805  148032  149010  149800  150761.0  151067.0
1  1993  157555  156266  154752  158979  160605  160127  162816.0  162506.0
2  1994  167518  169649  172766  173106  172329  174241  174781.0  177295.0
3  1995  182413  179488  181013  181686  183536  186081  185431.0  186806.0
4  1996  189135  192266  194029  194744  196205  196136  196187.0  196218.0

      SEP      OCT      NOV      DEC
0  152588.0  153521.0  153583.0  155614.0
1  163258.0  164685.0  166594.0  168161.0
2  178787.0  180561.0  180703.0  181524.0
3  187366.0  186565.0  189055.0  190774.0
4  198859.0  200509.0  200174.0  201284.0
```

```
[10]: df.shape
```

```
[10]: (30, 13)
```

I still find myself wanting to always see the data first and the shape just to check before and after changes to the dataset.

```
[14]: # Convert wide (YEAR + JAN..DEC) to long monthly time series

# Define months
months = [
    "JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC"]
month_map = {m:i+1 for i,m in enumerate(months)}

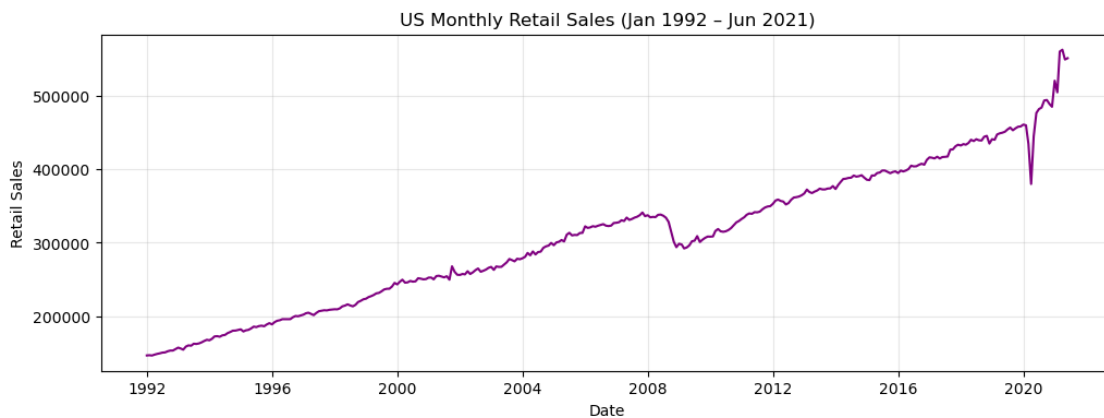
# Change format from human-readable to numbers
long = df.melt(id_vars="YEAR", value_vars=months, var_name="MONTH",
    value_name="SALES")
long["MONTH_NUM"] = long["MONTH"].map(month_map)
long["DATE"] = pd.to_datetime(long["YEAR"].astype(str) + "-" +
    long["MONTH_NUM"].astype(str) + "-01")

# Times series dictionary
ts = (long.dropna(subset=["SALES"])
      .sort_values("DATE")[["DATE", "SALES"]]
      .reset_index(drop=True))
```

```
[19]: # Verify changes
long.shape
```

```
[19]: (360, 5)
```

```
[21]: # Plot full series
plt.figure(figsize=(12,4))
plt.plot(ts["DATE"], ts["SALES"], color='purple')
plt.title("US Monthly Retail Sales (Jan 1992 - Jun 2021)")
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.grid(True, alpha=0.3)
plt.show()
```



```
[22]: test_start = pd.Timestamp("2020-07-01")
test_end = pd.Timestamp("2021-06-01")

train = ts[ts["DATE"] < test_start].copy()
test = ts[(ts["DATE"] >= test_start) & (ts["DATE"] <= test_end)].copy()

y_train = train.set_index("DATE")["SALES"]
y_test = test.set_index("DATE")["SALES"]

[23]: print("Train months:", len(y_train))
print("Test months:", len(y_test))
print("Test range:", y_test.index.min().date(), "to", y_test.index.max().date())
```

```
Train months: 342
Test months: 12
Test range: 2020-07-01 to 2021-06-01
```

```
[25]: import statsmodels.api as sm

model = sm.tsa.statespace.SARIMAX(
    y_train,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

res = model.fit(dispatch=False)
print(res.summary())
```

```
C:\Users\ayden\miniconda3\envs\book_env\lib\site-
packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency
information was provided, so inferred frequency MS will be used.
```

```
self._init_dates(dates, freq)
C:\Users\ayden\miniconda3\envs\book_env\lib\site-
packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency
information was provided, so inferred frequency MS will be used.
self._init_dates(dates, freq)
```

SARIMAX Results

```
=====
=====
Dep. Variable:                SALES    No. Observations:
342
Model:                SARIMAX(1, 1, 1)x(1, 1, 1, 12)    Log Likelihood
-3210.179
Date:                Sun, 15 Feb 2026    AIC
6430.359
Time:                12:04:57    BIC
```

```

6449.122
Sample:                                01-01-1992    HQIC
6437.855

                                - 06-01-2020
Covariance Type:                    opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.5570      0.083      6.676      0.000      0.393      0.721
ma.L1         -0.7234      0.079     -9.193      0.000     -0.878     -0.569
ar.S.L12       -0.2604      0.138     -1.884      0.060     -0.531      0.010
ma.S.L12       -0.2806      0.079     -3.556      0.000     -0.435     -0.126
sigma2        4.564e+07    7.07e-09    6.46e+15    0.000    4.56e+07    4.56e+07
=====
===
Ljung-Box (L1) (Q):                2.59    Jarque-Bera (JB):
19435.44
Prob(Q):                            0.11    Prob(JB):
0.00
Heteroskedasticity (H):            7.97    Skew:
-1.11
Prob(H) (two-sided):              0.00    Kurtosis:
41.42
=====
===

```

Warnings:

```

[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
[2] Covariance matrix is singular or near-singular, with condition number
7.8e+30. Standard errors may be unstable.

```

That is such a cool output. Some of these packages never cease to amaze me with how much information they provide with such little code. The speed is impressive too. I understand now why some datasets take forever to run, especially why Excel files take longer here in a Python environment vs converting them to CSV, but models seem to run pretty quickly, no matter what. I'm sure there are exceptions, but I'm still impressed.

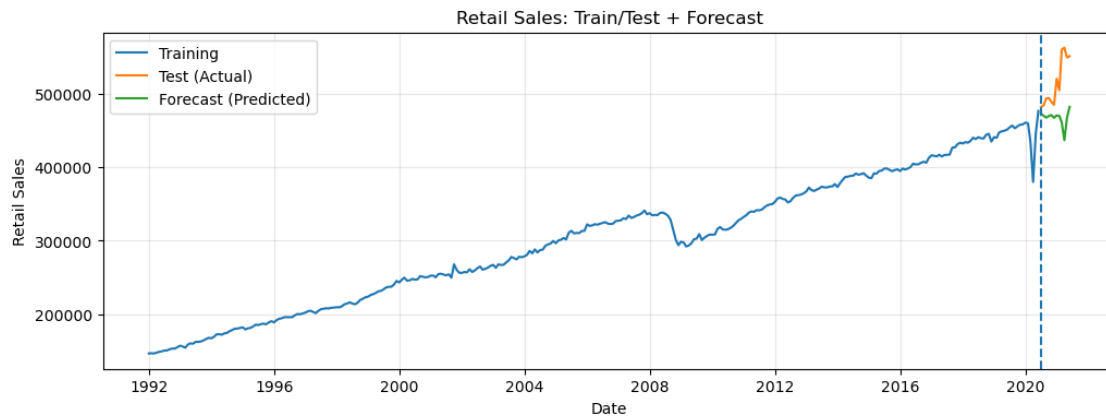
```

[26]: forecast = res.get_forecast(steps=12)
      pred = forecast.predicted_mean

      plt.figure(figsize=(12,4))
      plt.plot(y_train.index, y_train.values, label="Training")
      plt.plot(y_test.index, y_test.values, label="Test (Actual)")
      plt.plot(pred.index, pred.values, label="Forecast (Predicted)")
      plt.axvline(test_start, linestyle="--")
      plt.title("Retail Sales: Train/Test + Forecast")

```

```
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



```
[27]: from sklearn.metrics import mean_squared_error
import math

rmse = math.sqrt(mean_squared_error(y_test, pred))
print("RMSE:", rmse)
```

RMSE: 59819.07103846954

The RMSE is approximately 59,000, which means the model's predictions were off by about 59,000 (million dollars) per month on average during the July 2020 to June 2021 test period. While this may appear large, monthly retail sales values during this time range between roughly 400,000 and 600,000, so the error represents roughly 10–12% of total monthly sales.

The RMSE is likely at this level because the test period includes significant volatility caused by the COVID-19 pandemic. Retail sales during 2020 and 2021 experienced unusual shocks and rapid changes due to lockdowns and stimulus payments. Since the model was trained primarily on pre-pandemic data with more stable seasonal patterns, it struggled to fully capture these structural changes, resulting in higher prediction error.

[]: